Hexa



「開發效能提升實戰」

Claude Design 搭配自建 Skill, 團隊達到提升四倍產出

A-mi 吳品毅 & Jennifer 林杰伶

2026 Hello World Dev Conference | 10/14 17:00 | DE 會議室

2026





今天的兩位講者

阿咪 A-mi 吳品毅

敦謙國際 | 經理 / Scrum Master

Hexa 旅宿通路管理系統 產品負責人

台灣敏捷大賞 2026年度變革引導者、2025年度敏捷傳教士

致力於打造幫助人、而非取代人的系統

Jennifer 林杰伶

敦謙國際 | 資深 UI/UX 設計師

Hexa 產品設計：從需求、流程到介面

近期專注 AI × Design，把 AI 帶進產品開發

讓設計師更貼近開發

Hexa

今天你會帶走的三件事

1

AI 協作流程化

把團隊 SOP 與開發規範封裝成自建 Skill，減少 AI 產出偏離與人工修正的時間

2

產出倍增的關鍵

Claude Design 搭配自建 Skill 的架構邏輯，以及在真實專案落地推動4 倍產出

3

避坑指南與實務數據

第一手的導入痛點、指標衡量經驗與採坑教訓，縮短自家團隊的摸索期

Hexa

01. 開發效能提升？

2026



Hexa

近幾年，我們都在談「幾倍工程師」

每個人都在問：導入 AI 之後，效能提升了幾倍？



Hexa

但「幾倍」到底是在倍什麼？

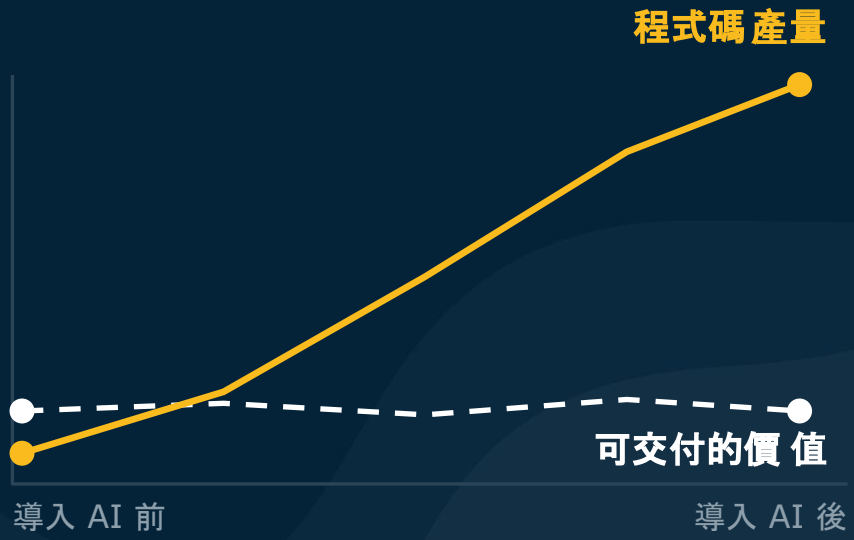
多寫了幾倍的程式碼？

多開了幾倍的分支？

多送了幾倍的 PR？

數字確實長大了 —

但投資回報率沒有跟著動。



Hexa

所以，我們先談談投資回報率（ ROI）

ROI ＝（回報 － 投入）÷ 投入

投入

授權費 · API 用量

學習曲線

整條流程重建的時間

回報

可以交付給客戶的價值

（不是程式碼行數）

中間那一段轉換率，才是我們真正要證明的東西

Hexa

然後……老闆關注的真的是 ROI 嗎？

其實老闆的問題更直接 —

1

公司可以得到什麼？

2

能多賺多少？

3

能建立什麼
別人追不上的優勢？

Hexa

「關我屁事？」這是許多 RD 心中的想法

每天需求做不完、bug 要處理，

上面的需求要開發……

還要我怎樣！？



Hexa

結果……

「好啊，反正我很忙，現在又有 AI，那都給 AI 做吧。」

5,000

行程式碼 ／ 天

「我要當一天提交 5,000 行程式碼的 Vibe Coder。」

Hexa

然後，這些開始出現

「為什麼這裡壞掉了？」追追追 — 啊，AI 改的。

「我也不知道為什麼壞了，我先改回來。」

「啊啊啊，那個新功能壞掉了，我叫 AI 修正。」

「AI 說修好了……但試出來還是壞的。為什麼？」

Hexa

問題不在 AI，在於沒有人定義「什麼叫做好」

程式碼產量，變成唯一看得見的指標

沒有定義產出，AI 就只是把原本的混亂放大

跑得更快了，但不知道要往哪裡跑

Hexa

02. 產出提升

2026

Hexa

什麼叫產出提升？

✕ 不是產出

我寫的程式碼變多了

✓ 才是產出

持續交付給客戶的價值

提升這個「可交付的價值」，才有意義

Hexa

那……到底要提升「哪一種」產出？

貼合產品、對客戶有價值的產出，才是有意義的

AI 做得出來，但飯店用不到

- 蒐集新聞
- 整理會議記錄
- 整理筆記

飯店用戶真正要的

- 官網版型
- 訂房流程
- 數據分析

Hexa

讓我們回到團隊成員身上……



Hexa

回到那一句 —「關我屁事？」

要回答的其實不是「你為什麼不用 AI」，

而是「用了 AI，對你有什麼好處」。

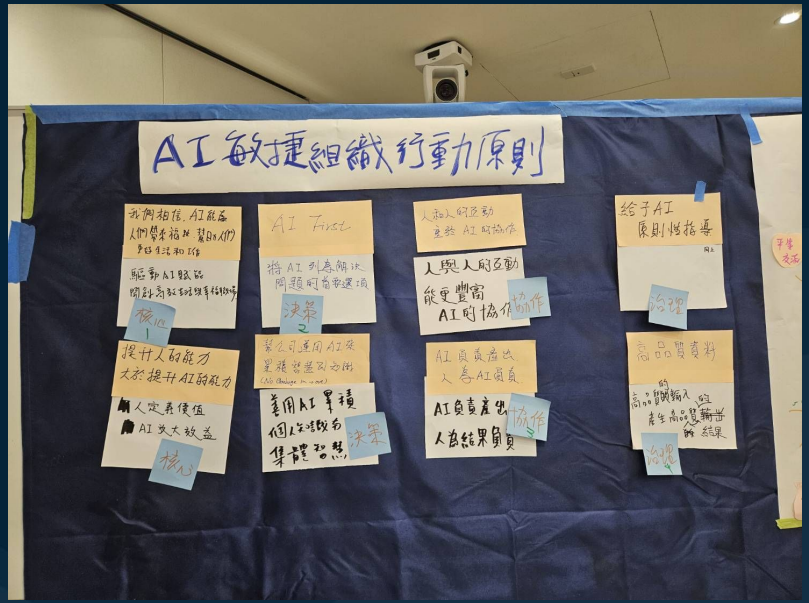
Hexa

對團隊來說，不是提升，是解放

2026 AI 敏捷組織行動原則

人定義價值

AI 放大效益



Hexa

現在的 AI 世代，人的價值在於微決策

過去已經知道怎麼做的事、低熵的事，就該交給 AI 處理

人要做的是定義價值，然後持續決策「下一件更有價值的事」



↻ 然後再回到第一步 — 這就是人在迴圈裡的位置

Hexa

所以，什麼交給 AI，什麼留給人

交給 AI｜低熵

規則已經知道

流程重複發生

產出打開就能驗收

差異在風格，不在邏輯

留給人｜高熵

定義要解決什麼問題

決定先做哪一個

判斷什麼叫做「好」

承擔取捨的後果

Hexa

當市場、當環境都在繞著 AI 跑，
我們要讓團隊知道為何而戰

Hexa

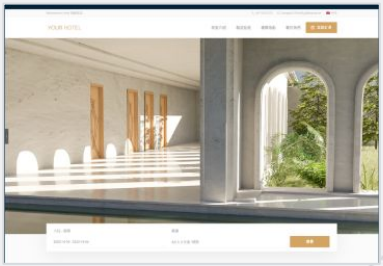
03. 定義產出與價值

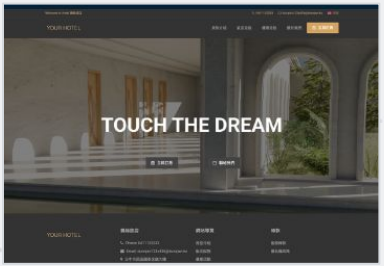
2026

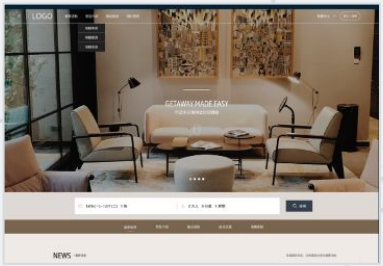
Hexa

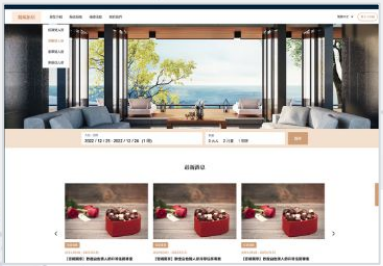
Hexa 是提供飯店多種版型的官網系統

敦謙國際：台灣最大飯店集團，50 多家自有飯店、系統大多自研。Hexa 是集團的旅宿通路管理系統，官網版型是其中一條產品線

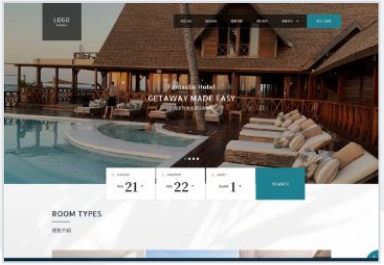


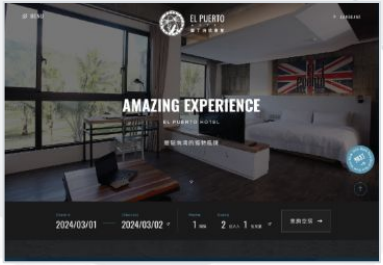


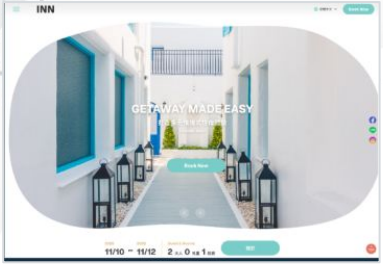












Hexa

年度目標也是如此

O2：透過 AI、數據分析與功能升級打造智能化的市場競爭優勢	KR1. 成功開發並上線 AI Agent (如：智能客服 Agent、後台操控 Agent)。
	KR2. AI Agent 功能的用戶滲透率達到 **15%**。
	KR3. 持續推進官網版型豐富程度到20套。

我們安排一個前端、一個設計，每個月出 1 版，

持續推進這個 KR。

Hexa

先把這件事的工時結構拆開來看

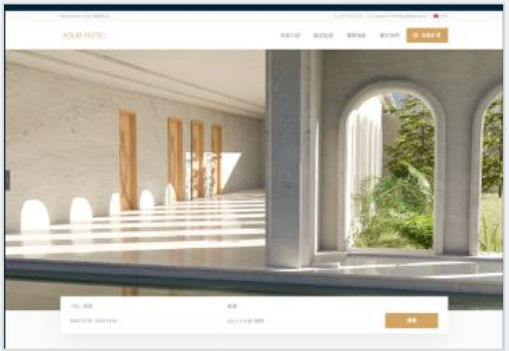
設計｜1 個月 → **前端｜1 個月**

2 人月	8 週	1 套／月
完成一套版型的人力投入	從設計到上線的交付週期	以接力方式排程的穩定產出

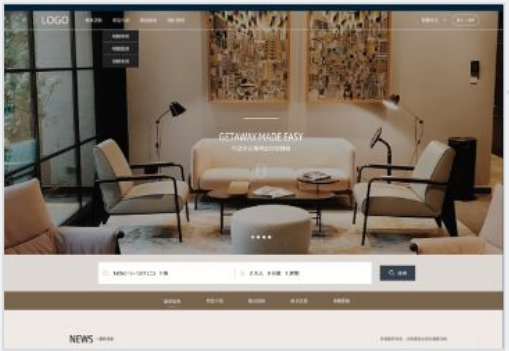
Hexa

這是一個重複、常態性質的工作

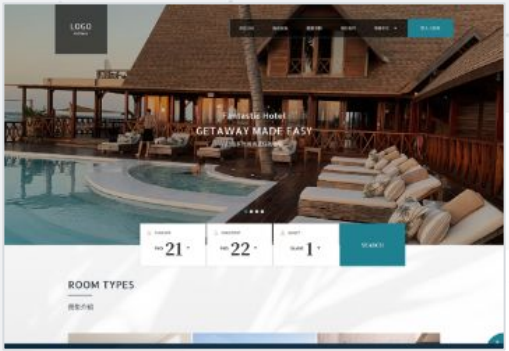
但不是每個月產出一樣的東西，而是各種不同的版型設計與風格。



第 1 個月



第 2 個月



第 3 個月

Hexa

為什麼這件事，正好適合交給 AI

低熵	流程已知、規格明確，每一套的差別在風格，不在邏輯
可驗收	做出來好不好，打開瀏覽器就看得見，不需要爭論
高重複	一年要推到 20 套，每一套省下的時間都會複利





Hexa

所以我們在這樣的 case 上導入

Claude Design
搭配自建 Skill



Hexa

04. Claude Design
搭配自建 Skill

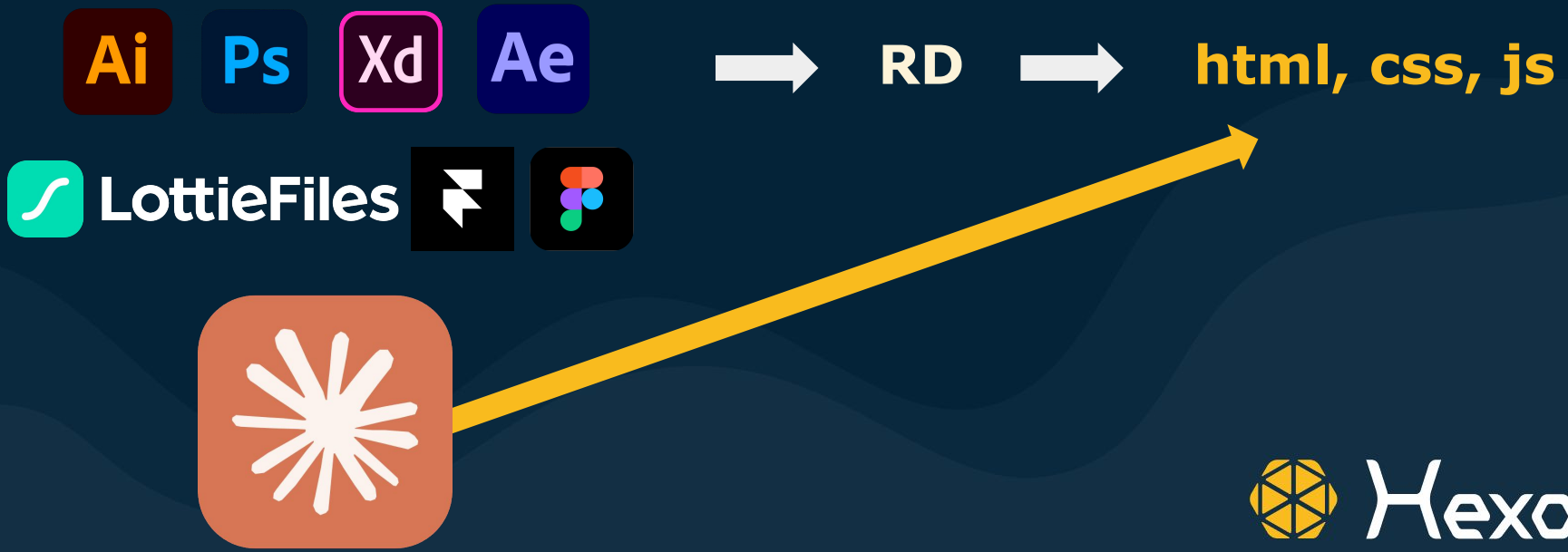
2026

Hexa

工具的選擇？

介面＆動態設計

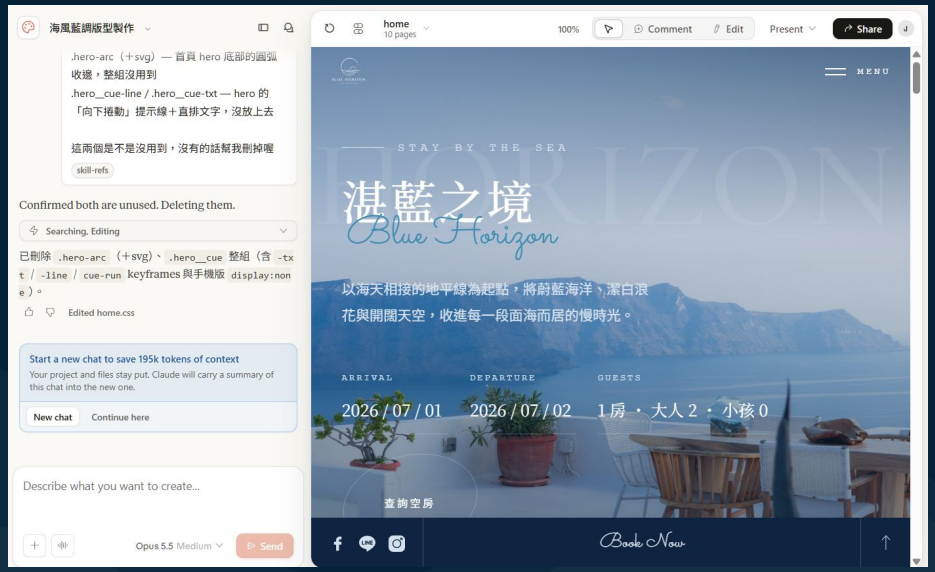
團隊可用的靜態檔

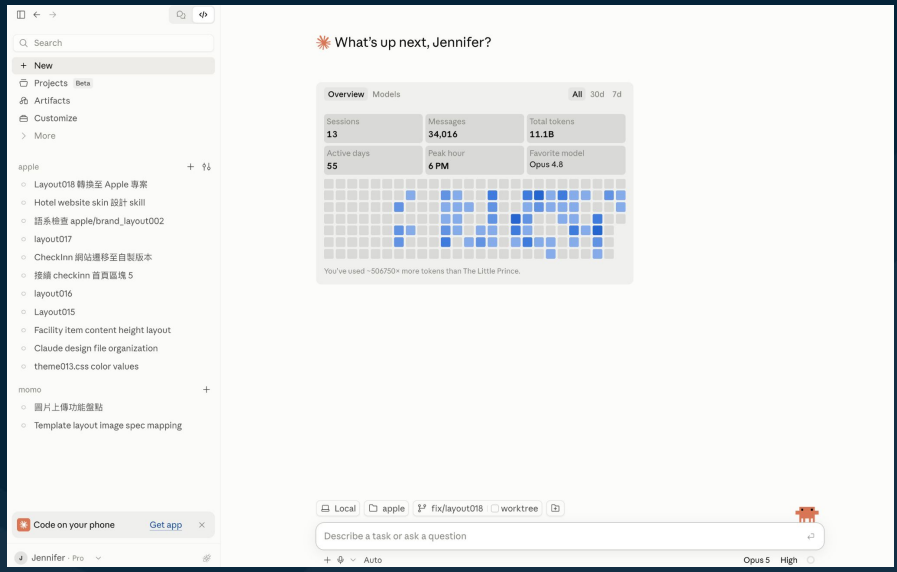


Claude design vs Claude code

不會畫圖 | 樣式不要用edit改 | 線上

設計能力不如 design | 本地端





Hexa

初次導入的蜜糖

設計師	前端	測試
• Figma 設計 • 元件、功能、動態文件說明	• 切版 • 串接資料	• 版面、多語系檢查

Claude Design	導入前端專案	測試
規格書、設計＋切版	**Skill 搬遷＋串接**	• 多語系檢查

設計20天+切版10天 □ **15天**　10天 □ **5天**　**1天**

40天 □ **21天**

Hexa

我怎麼做 導入專案的轉換 skill？

目標	設計原始檔合理拆分元件、導入專案並串接資料。
規格	一開始會拿到的設計原始檔 想要的成果：已有範例，請 claude code 進專案研讀
規劃	請 AI 規劃 skill 架構，並設置優化規範，確保後續優化時結構乾淨
測試	請 AI 模擬測試，自行優化
實際使用	實際使用，觀察反覆出現的錯誤，持續優化

Hexa

Skill: 導入前端 Angular 專案＋資料串接

Skill 節錄

Claude Design → Angular 官網轉換器

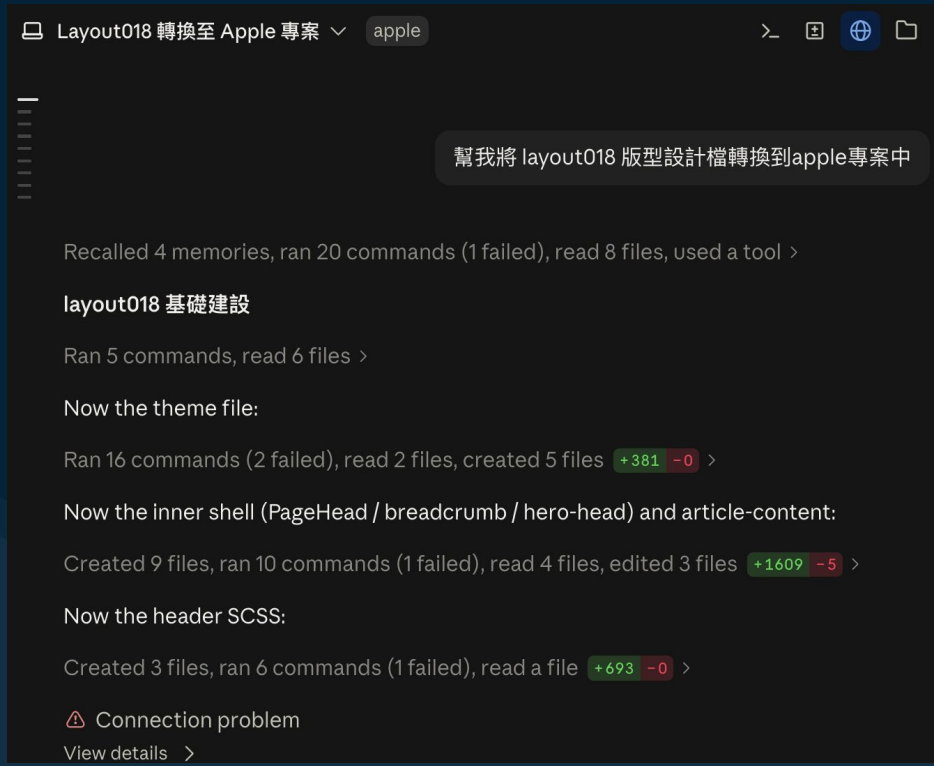
目標

把 Claude Design 匯出的一包官網版型（每頁獨立的 HTML、共用＋各頁 CSS、JS、assets），在**不改動畫面呈現**的前提下，重構成本團隊的 Angular 18 官網專案結構，統一 coding style，並把寫死的內容接上資料來源（透過 `extends` base class），讓成品接近可直接運作，前端只需收尾。

核心原則（最高優先，凌駕後面所有細節）：

1. **忠實還原設計，含動態效果**：本 skill 轉**各種不同版型**，一律照「這次的設計稿」還原——含 RWD，也含**動態/過場**（進場、捲動浮出、輪播切換、視差、轉場都是設計的一部分，不可只搬靜態樣式）。
2. **降低維護成本**：優先用共用元件；重複結構抽 component、重複樣式集中；**沿用優於新寫**（優先 `extends` base / 既有 helper、helper 不夠先擴充再引用）；**建立與清理對稱**；**不用** `!important`（覆蓋共用元件改用「提高特異度」）。
3. **只改被要求的、不確定先問**：若忠實還原會導致難以維護，**先提問或提建議，不可自行簡化或擅改樣式**。

實際對話截圖



Skill 檢查清單：確保與設計稿相同

Skill 節錄：檢查清單

- [ ] 「忠實還原 vs 難維護」取捨已與使用者確認，未自行簡化/擅改。
- [ ] 完工反向對照已做：拿設計稿逐元素走一遍、對照「先輸出的清單」逐項打勾，無遺漏。
- [ ] 畫面與 RWD 與原稿一致。

Skill 節錄：

強調依照設計稿 產出，避免被其他版型作法影響

保真度目標：以「本版型設計稿」為準，盡量 1:1 重現

- **每個版型風格不同，動畫保真度對齊的是「這份設計稿本身」，不是其他版型。** 不要因為別的版型 把某效果簡化掉，就跟著降級——以本次原稿實際呈現為準。
- **實作技術則對齊專案慣例**：優先用專案**現有的 gsap 能力**重現效果，而不是照搬原生 JS（IntersectionObserver、`mousemove` + rAF、`setInterval`）。gsap 版本統一、效能與維護更好。
- 這兩件事不衝突：視覺結果忠於設計稿，實作手段忠於專案技術棧。

Skill 節錄：自動 產出執行清單 ☐ 實作 ☐ 驗收準則

轉換前：先「輸出」動態/互動清單（強制，看得到才擋得住漏）

> 這步是必須產出的動作，不是「心裡想一想」。複雜頁最常見的失誤，就是把重心放在「結構跑起來 + 資料綁上」，而沒有逐元素盤點動態——因為沒盤點，根本不會意識到缺了什麼，於是連「漏了」都不知道、也不會回報。解法：把盤點變成一份看得到的清單。沒有先產出清單，不得開始寫該頁。

對該頁**逐元素**掃過設計稿的 HTML/CSS/JS，用**固定分類**逐項列出（用固定分類是為了逼出最常漏的「狀態類微互動」）：

- **進場 / 捲動浮出**：reveal、stagger、視差。
- **hover**：位移、換色、放大、底線…（卡片、連結、按鈕、圖片）。
- **active / selected**：tab 選中換色、目前頁/目前項標記、輪播目前 dot。
- **focus / 表單狀態**：輸入框、可聚焦元素。
- **transition**：展開/收合、淡入淡出、顏色過渡。
- **輪播行為**：peek 露邊 / loop / autoplay / dots 樣式 / 箭頭 / 邊界禁用。
- **空狀態 / 條件顯示**：無資料、選填區塊。
- **手勢**：swipe、long-press。

Hexa

Skill 檢查結果與設計稿不符 □ 自動回報

Layout018 轉換至 Apple 專案 apple

⚠️ 與設計稿的差異（請確認）

1. **燈箱**：設計是自製的極簡單張版（只有圖＋右上×）→改用共用 `ImageCarouselPopupComponent`。差異＝外觀與操作走共用元件（含左右切換）。
2. **地圖**：`.map__frame` 的**虛線**佔位框改成實線細框，內容換成 `<app-google-map>`（新增 `map-banner-config.ts` 米杏 / 赭陶色盤）。理由：虛線在本版是「還沒發生」的語彙，地圖已是實物。
3. **日期 / 人數選擇器**：票券 Stay / Guests 接共用 `app-date-picker-web` / `app-people-filter-web`（電腦就地展開、手機底部彈窗），外觀為共用元件預設。
4. **票券存根編號**：設計寫死 `No. 93721`，後台無此欄位 → 首頁改用入住日組成的序號（如 `260625`），rooms-detail 用當前房型 `Room 0X`。
5. **頁尾宣言短文**：後台沒有專屬欄位 → 綁 `about.subTitle`（飯店簡介）。簡介較長時頁尾會變長。
6. **推薦景點裝飾主圖不串後台**：`IHotelTemplateImages` 只有 about / facility / promotion / room，**沒有 attraction** → 一律用預設圖 `img_attraction_banner.jpg`。
7. **景點圖上白框位置**（`--fx/--fy/--fw/--fh`）：設計是四張各寫死 → 改四組循環表依索引取模，任意筆數都成立。
8. **弧線刻度顏色**改走 `--dial-*` CSS 變數（父層設變數，不用 `::ng-deep` 穿透）；視覺與設計一致。
9. **設施頁回頂鈕不隨紙面訂房帶翻墨色**（側欄仍照設計動態翻）。理由：紙面帶捲到頂列時畫面下緣其實是深色頁尾，墨色鈕反而看不見。
10. **首頁房型卡的 `data-par` 視差實際無效**——設計稿 `.room-card.reveal.is-in { transform: none }` 會蓋掉視差位移；**照原樣保留、沒有自行「修正」**。
11. **捲動改用團隊慣例的 `.scroll-area` 容器**（設計是 window 捲動），所有捲動監聽 / 視差 / sticky 改對此容器；sticky top 值沿用設計數值。
12. **視差**由手刻 rAF 改用 gsap ScrollTrigger 當捲動觀察者，公式與字面值（`mid × speed`）完全照抄；**捲入浮現維持設計稿的 IntersectionObserver**（rootMargin −18% / threshold .12 / 延遲 0.12+i×0.16 照抄）。
13. **字距語系規則**放在外殼 scss 的 `:host[lang^="…"] .…text(…) ::ng-deep`，allowlist 照抄設計稿——日後新增等寬 / 英文小標類別要同步維護那份清單。

這樣就夠了嗎?

怎樣可以更快更好？

好煩，不想一直重複講規格
怎麼同樣的問題一直發生
吼！我前面不是說過了嗎?



Hexa

第二次導入

Claude Design 規格書、設計＋切版	**導入前端專案** **Skill 搬遷＋串接**	**測試** • 多語系檢查
Claude Design **Skill 導入規格、設計要求、 coding style、交接文件 ...**	**導入前端專案** **Skill 搬遷＋串接**	**語系檢查** **Skill 多語系轉換**

30天 □ 15天 □ **5天**　　10天 □ **5天**　　1天 □ **忽略不計**

40天 □ 21天 □ **10天**

Hexa

我怎麼做設計 skill？

目標	設計版型。欄位資料固定，只修改風格 **(定義何謂風格？)**
規格	提供設計檔(html, css 等)、規格書、設計前必要的資料蒐集。 coding style、產出檔案架構、TODO註解、交接文件 (版型設計說明)
規劃	請 AI 規劃 skill 架構，並設置優化規範，確保後續優化時結構乾淨
測試	請 AI 模擬測試，自行優化
實際使用	實際使用，觀察反覆出現的錯誤，持續優化



Skill: 詳細定義何謂風格、強調獨特性

Skill 節錄

核心哲學（最高原則，凌駕細節）

1. **結構與風格徹底分離**。骨架（HTML 語意、共用區塊、內容槽位、檔案慣例、JS 行為 DOM 契約） 由本 skill 固定；其餘全是風格，可整包替換。風格分成兩層，**兩層都要做，缺一不可**：·**視覺 token 層**（色/字/間距/圓角/陰影/動效）→ 收斂在 `theme.css`，換值即換色系。·**排版層**（構圖、對齊、區塊排列、header/footer 組織、卡片形制、標題設計）→ 寫在各頁與 共用 CSS 裡，**必須重新設計**。⚠️**只換 token、沿用同一套排版 ≠ 換皮**，那只是同一個版型換顏色。設計差異主要住在排版層。
2. **本 skill 本體只寫「角色名」與「規格」，絕不寫死任何色碼、字體名、特定動效或美學語彙。** 之所以這樣，是因為寫死任一種美學，下次換皮就換不動、還會污染產出。具體值只能出現在 `theme.css`（本版填）與 head 的字體 link，其餘一律 `var(--…)`。
3. **不複製任何特定版型的風格。** 每次的風格一律來自**當次使用者的風格 brief**。骨同皮異的意思是： 同一個 hero/rooms/footer 槽位，可以長成天差地別的樣子——例如同一個 hero，A 版可能是滿版壓暗 攝影＋襯線置中大標，B 版可能是大量留白＋細線分隔＋小標靠左，C 版可能是拼貼式多圖＋無襯線。 **槽位相同、呈現不同**，這就是換皮。設計時從當次 brief 生成風格，不要臨摹任何既有成品。
4. **只產靜態 demo，資料綁定交給下游。** 選單、房型卡、消息等一律用標準假資料寫死，好讓每頁都有畫面可看。任何「串接後台資料」屬 `design-to-angular`，本 skill 不碰。

設計決策方法

以下是做設計判斷時的通用方法，適用每一頁、每一個區塊。

預設解檢查（本 skill 最常被違反的一條）

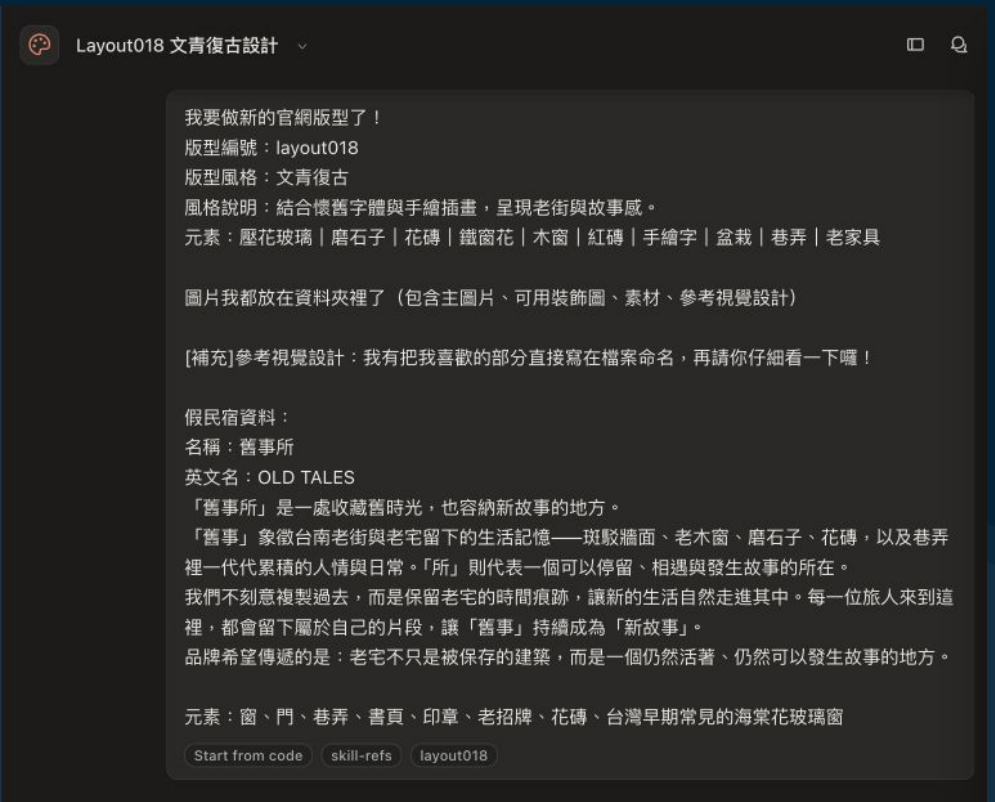
最先浮現在腦中的做法，通常不是設計，是預設值。 圓角、三欄式 footer、hover 放大圖片、置中的小字＋大標——這些會不假思索地出現， 因為它們是通用網頁 UI 的慣例，不是從本版風格推導出來的。

判準只有一句：**把它原封不動搬到另一個風格的版型上，會不會違和？ 不會違和 → 它沒抓到本版風格，要重做。**

Hexa

Skill: 風格元素提取

設計師:提供資料



Skill:給出風格元素提取結果

三、字體角色

- `--f-title` ：**Noto Serif TC**（700）— 中文大標、直排標題
- `--f-body` ：**Noto Serif TC**（400）— 中文內文
- `--f-en` ：**EB Garamond** — 英文標題 / 英文說明，復古書刊襯線
- `--f-mark` ：**Courier Prime**（等寬）— 編號、日期、規格標記、圖說。這支是「檔案 / 目錄卡」感的來源

四、風格密度：簡約｜節奏：中

裝飾量克制，靠留白、對齊軸、位置關係製造吸引力。動效 token 三階：`--dur .28s` / `--dur-slow .6s` / `--dur-slower 1.1s`、`--delay .08s`、`--ease cubic-bezier(.25,.6,.2,1)`。捲入浮現只做位移＋淡入，不做縮放彈跳。

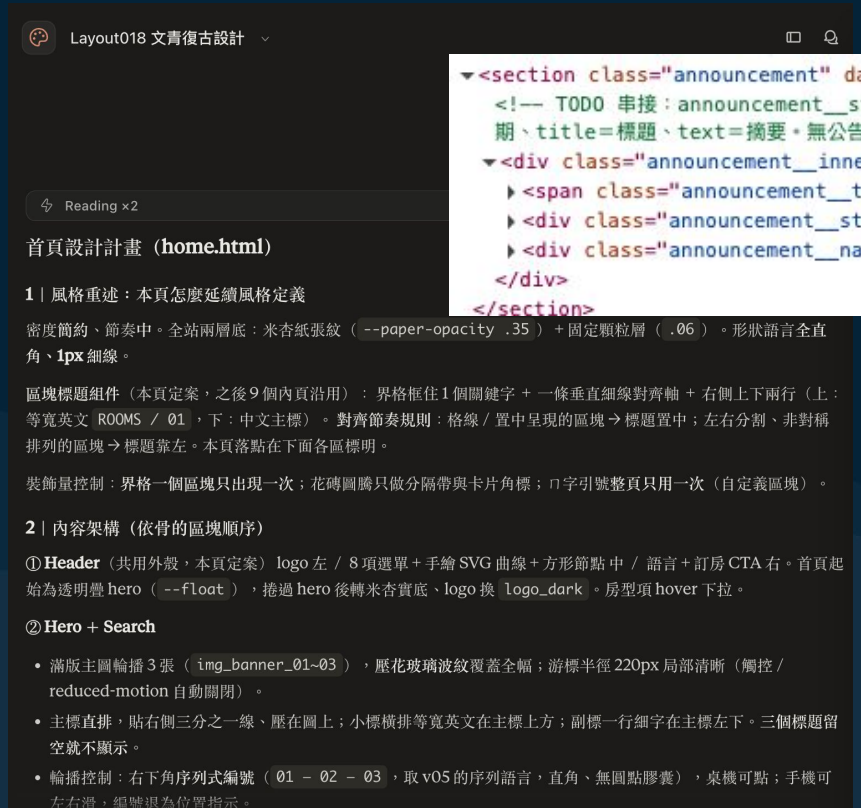
五、五個貫穿全站的辨識元素（都有規則，不是漂浮裝飾）

1. **界格（米字格）**：只用在「單一主角字 / 編號」上——方框＋內部細對角與中線，框住 1 個字或 2 位數字。規則：一個區塊最多一處，用來標示該區塊的起點。
2. **壓花玻璃波紋**：只覆蓋在圖片上（不覆蓋文字）。規則：波紋是圖片的「窗玻璃」，所以只出現在有邊框的圖上；hero 全幅、內頁圖為左右兩側 15% 漸弱。
3. **ㄇ字引號框**：只框引言／摘要（品牌宣言、房型引言、景點介紹首段），上下兩道開口相對的ㄇ形細線。規則：框內只放置中短文，不放圖也不放按鈕。
4. **細框＋引出線圖說（v09）**：圖片外一圈 1px 細框，框外一條短直線引出等寬字圖說（房型名 / 設施樓層 / 景點方位）。規則：引出線一律水平，長度固定 40px。
5. **花磚圖騰細節**：只做分隔與角落——區塊間分隔帶（單行小尺寸重複）、卡片左上角 24px 角標。規則：不當大面積底紋（會蓋掉留白）。

形狀語言：直角。所有元件（按鈕、箭頭、dots、回頂、標籤）一律 0 圓角＋1px 細線；只有界格是有內部結構的方框，不讓其他元件用相近形狀搶它。

Skill: 設計＋切版＋串接註解

動工前，先提出頁面設計計劃，避免 Token 浪費



TODO 串接註解

```
<section class="announcement" data-screen-label="Announcement">
  <!-- TODO 串接：announcement__stack 內 announcement__item 依「公告 / 置頂消息」資料迴圈輸出（輪播，取前 N）；每筆 period＝日期、title＝標題、text＝摘要。無公告時整條 ribbon 隱藏。 -->
  <div class="announcement__inner">  flex
    <span class="announcement__tag">…</span>  flex
    <div class="announcement__stack">…</div>
    <div class="announcement__nav">…</div>  flex
  </div>
</section>
```

Hexa

Skill: 交付設計檔

提供給下階段的交接文件

DOCUMENTS

圖片尺寸建議.md Document	over a week ago
設計定案.md Document	over a week ago
CLAUDE.md Document	over a week ago
article-toc-說明.md Document	over a week ago

檔案結構

FOLDERS

assets Folder	—

PAGES

home.html HTML page	over a week ago
room-list.html HTML page	over a week ago
article.html HTML page	over a week ago
room.html HTML page	over a week ago
promotion.html HTML page	over a week ago
promotion-list.html HTML page	over a week ago
privacy-policy.html HTML page	over a week ago
facilities.html HTML page	over a week ago
attraction.html HTML page	over a week ago
about.html HTML page	over a week ago

google 翻譯好爛

一直貼到 ChatGPT 翻譯好累

Hexa

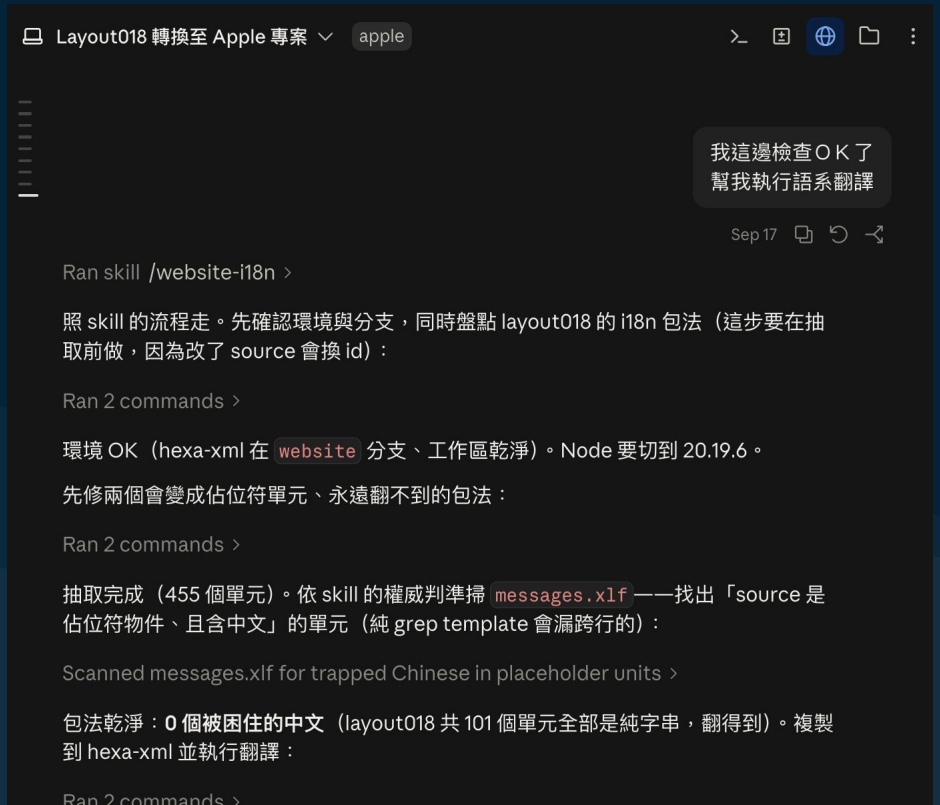
多語系自動翻譯 skill

目標	依照**實際頁面**查看翻譯結果並修正優化
規格	實際操作一次，再請claude code 整理流程變成 skill
規劃	請 AI 規劃 skill 架構，並設置優化規範，確保後續優化時結構乾淨
測試	請 AI 模擬測試，自行優化
實際使用	實際使用，觀察反覆出現的錯誤，持續優化

Hexa

Skill: 多語系

對話截圖



Skill 回覆修正結果



原文	機翻錯在哪	已改為
存根	ko 그루터기 ＝樹樁、th ตอ ＝樹樁	ja 半券 / ko 보관용 / vi cuống vé / th ต้นขั้ว / id potongan tiket
樓層	ja 床、ko 바닥、vi sàn nhà、th พื้น、tl sahig 全是地板	階 / 층 / Tầng / ชั้น / Palapag
房（量詞）	th บ้าน ＝房子	ห้อง
人（量詞）	th ประชากร ＝人口	คน
窗戶	ja ウィンドウ ＝電腦視窗	窓
衛浴	ko 화장실 ＝廁所（同檔的「衛浴設備」卻用 욕실，不一致）	욕실
挑一個好日子住下來吧。	「住下來」被當成定居：th เริ่มต้นชีวิตคู่ ＝開始夫妻生活、vi ổn định cuộc sống、ko 정착	全數改成「來住一晚」語意
館外的部分…	「館」被當成博物館（全語系）	改成「我們這裡 / 住宿之外」
每一處的位置與使用方式	「每一處」被當成每項物品	改成「每個空間」
喜歡我們的空間	全語系自行加上「具而	改回「來住」

那...

有沒有甚麼，快不了？

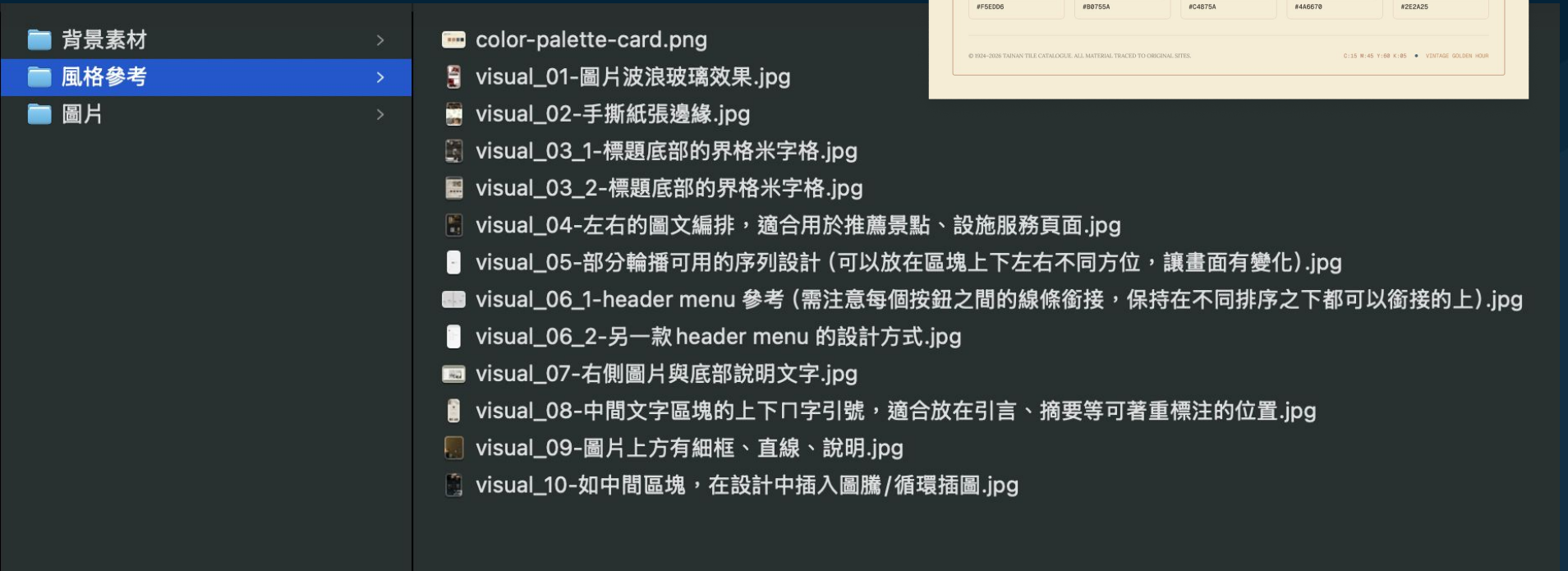


Hexa

無可避免的設計前置作業

靈感、素材蒐集、風格確認





持續觀察微小的 "重複發生"

都是優化的可能性

Hexa

05. 成效

2026

Hexa

導入前 vs. 導入後

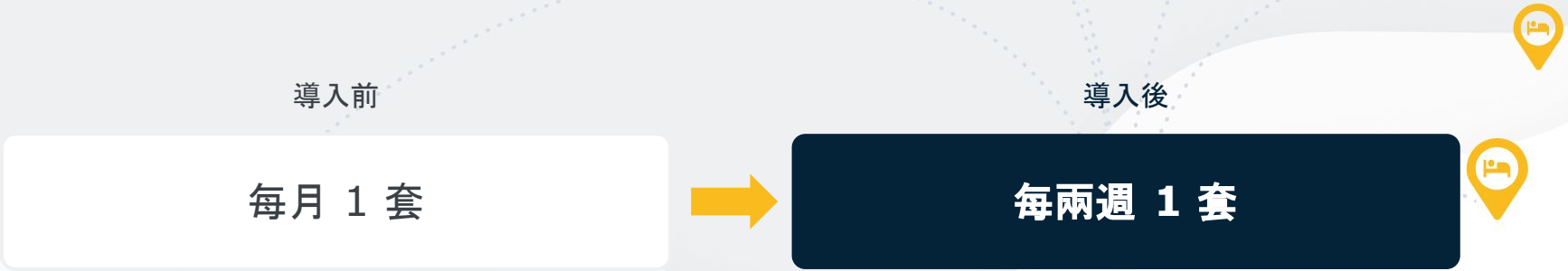
	導入前	導入後
一套版型的人力投入	2 人月	**0.5 人月**
從設計到上線的交付週期	8 週	**2 週**
每月穩定產出	1 套	**2 套**

人力投入 ÷ 4 ｜ 交付週期 ÷ 4 → 4 倍產出

Hexa

解放 ①:交付節奏

我們以約每兩週一次的節奏釋出版型



解放 ②:前端

前端完全從重複性工作中解放,
轉去優化訂房與數據分析系統

導入前

重複性的版型工作

導入後

訂房與數據分析系統

Hexa

解放 ③:設計

設計可以更自由地安排版型的創作與時程

導入前

配合前端的檔期

導入後

自由安排創作與時程

Hexa

解放 ④:思考力

AI 的價值在於解放人的思考力 —
不用把時間花在重複且低 熵的工作上

導入前

時間花在低熵的工作

導入後

有時間思考下一步

Hexa

06. 避坑指南

2026

Hexa

我們踩過的三個坑

坑一

把「產出」等於「程式碼量」

數字一路長大，可交付的價值卻沒有動

坑二

沒有驗收標準就交給 AI

「AI 說修好了」，但試出來還是壞的

坑三

挑錯題目

做出一堆對使用者沒有價值的功能





Hexa

如果你要在團隊推這件事

1. 先定義「什麼叫做好」，再決定要自動化什麼
2. 從低熵、可驗收、高重複的工作開始，別一上來就動核心系統
3. 用交付週期與人力投入衡量，不要用程式碼行數

Hexa

人定義價值，
AI 放大效益。

Hexa



Thank you for listening.

A-mi 吳品毅 & Jennifer 林杰伶 ｜ 敦謙國際 Hexa

Hexa